



نام درس: سیستم های توزیع شده

اعضای گروه:

فرشته باقری - ۸۱۰۱۰۰۰۸۹

الهه خداوردی - ۸۱۰۱۰۰۱۳۲

محمد امانلو - ۸۱۰۱۰۰۰۸۴



تمرین شماره ۲

مقدمه

در این پروژه، هدف ما طراحی و پیاده‌سازی یک سرویس ساده اما قابل اطمینان کلید/مقدار (Key/Value Server) با قابلیت کار در محیط‌های شبکه ناپایدار بود. سیستم پیاده‌سازی شده باید بتواند تضمین کند که هر درخواست نوشتن حداقل یک بار و حداکثر یک بار روی سرور اجرا شود و از نوشتن دوباره داده‌ها جلوگیری کند. همچنین، این سرویس باید خطی‌سازی پذیر (linearizable) باشد تا رفتار مشابه یک سرور single thread با اجرای ترتیبی درخواست‌ها داشته باشد. در ادامه، مکانیزم قفل توزیع شده‌ای نیز روی این سرویس طراحی شد تا چند کلاینت به صورت هماهنگ روی یک ماشین بتوانند به طور همزمان و بدون تضاد به منابع دسترسی داشته باشند.

این پروژه فرصتی بود تا مفاهیم پایه‌ای رایانش توزیع شده از جمله semantics once-most-at linearizability و پیاده‌سازی قفل توزیع شده را به طور عملی تجربه کنیم. همچنین، چالش‌هایی مانند شبکه ناپایدار، گم شدن پیام‌ها، تاخیرها و بازفرستادن درخواست‌ها (retry) نیز در طراحی و پیاده‌سازی در نظر گرفته شد تا سیستم حاصل، در شرایط واقعی پایدار و قابل اتکا باشد.

شرح پروژه

در این پروژه، یک سرور کلید/مقدار ساده طراحی و پیاده‌سازی شد که از طریق فراخوانی‌های RPC، کلاینت‌ها قادر به ذخیره‌سازی (Put) و بازیابی (Get) مقادیر هستند. سرور از یک ساختار map در حافظه برای نگهداری جفت‌های کلید-مقدار به همراه ورژن هر کلید استفاده می‌کند.

هر درخواست Put شامل کلید، مقدار و ورژن ارسال می‌شود و تنها در صورتی پذیرفته می‌شود که نسخه ارسالی با نسخه فعلی سرور برای آن کلید مطابقت داشته باشد. این مکانیزم تضمین می‌کند که عملیات نوشتن به صورت اتمیک و متوالی انجام شده و از بازنویسی‌های ناهمزمان جلوگیری شود. همچنین، ورژن هر کلید پس از هر به‌روزرسانی یک واحد افزایش می‌یابد.

برای هماهنگی دسترسی همزمان، در سمت سرور از mutex استفاده شده است تا تنها یک thread بتواند همزمان تغییرات روی یک کلید را انجام دهد. این امر باعث می شود که سرور رفتار خطی سازی پذیری را حفظ کند. در سمت کلاینت، مکانیسم های retry و backoff برای مواجهه با خطاهای احتمالی شبکه و از دست رفتن پیامها پیاده سازی شده است. کلاینت ها با تولید شناسه های یکتا برای هر درخواست Put اطمینان حاصل می کنند که درخواست ها حداکثر یک بار روی سرور اجرا می شوند.

بخش بعدی پروژه به پیاده سازی قفل توزیع شده اختصاص داشت که از سرویس کلید/مقدار به عنوان پایه استفاده می کند. این قفل توزیع شده به کلاینت ها اجازه می دهد تا با حفظ قابلیت خطی سازی، به صورت ایمن و هماهنگ به منابع مشترک دسترسی داشته باشند. برای این منظور، روش هایی مانند شناسه گذاری کلاینت ها و مکانیزم های بازفرستادن درخواست با backoff مناسب به کار گرفته شده اند.

در نهایت، سیستم طراحی شده با اجرای تست های جامع تحت شرایط شبکه های قابل اطمینان و ناپایدار مورد ارزیابی قرار گرفت. نتایج تست ها نشان داد که سیستم قادر است در شرایط مختلف عملکرد صحیح، هماهنگ و قابل اعتمادی داشته باشد و چالش های ناشی از ناپایداری شبکه را به خوبی مدیریت کند.

توضیحات ساختار کد کلاینت

در این بخش، به تحلیل ساختار کد کلاینت پرداخته می شود که وظیفه ای اصلی آن مدیریت ارتباط با سرور کلید/مقدار از طریق RPC است. این کد به طور خاص برای پیاده سازی دو عملیات اصلی یعنی Get و Put طراحی شده است.

ساختار کلی کد

ابتدا، ما یک ساختار Clerk تعریف کرده ایم که شامل دو فیلد اصلی است:

□ `clnt`: که یک اشاره گر به ساختار `Clnt` از بسته `tester` است و مسئول مدیریت ارتباطات RPC با سرور می باشد.

□ `server`: که آدرس سرور (string) را ذخیره می کند تا در طول عملیات `Get` و `Put` برای فراخوانی ها استفاده شود.

ساختار کلاینت به گونه ای طراحی شده که از طریق RPC، درخواست ها را به سرور ارسال کرده و پاسخ ها را دریافت می کند.

تابع MakeClerk

تابع MakeClerk وظیفه‌ی ساخت یک نمونه از Clerk را بر عهده دارد و فیلدهای clnt و server را مقداردهی اولیه می‌کند. در اینجا هیچ منطق اضافی دیگری در نظر گرفته نشده است. به این صورت است که یک کلاینت جدید برای ارتباط با سرور ساخته می‌شود:

```

1 func MakeClerk(clnt *tester.Clt, server string) kvtest.IKVClerk {
2     ck := &Clerk{clnt: clnt, server: server}
3     // You may add code here.
4     return ck
5 }
```

تابع MakeClerk مسئول ایجاد و مقداردهی اولیه یک نمونه از ساختار Clerk است که نماینده‌ی کلاینتی است که می‌خواهد از طریق فراخوانی‌های RPC با سرور کلید/مقدار تعامل داشته باشد. این تابع به عنوان نقطه شروع برای استفاده کلاینت در ارتباط با سرور عمل می‌کند و ورودی‌های آن شامل اشاره‌گری به ساختار Clnt است که مدیریت ارتباطات RPC را بر عهده دارد و همچنین رشته‌ای که آدرس یا شناسه سرور کلید/مقدار را مشخص می‌کند. در این تابع یک نمونه جدید از ساختار Clerk ساخته شده و فیلدهای آن شامل clnt که مسئول ارتباط RPC است و server که آدرس سرور را نگهداری می‌کند، مقداردهی اولیه می‌شوند. این تابع صرفاً وظیفه مقداردهی اولیه ساختار کلاینت را دارد و منطق پیچیده‌ای در آن اجرا نمی‌شود، اما در صورت نیاز می‌توان تنظیمات بیشتری مانند تولید شناسه یکتا برای کلاینت یا پارامترهای کمکی دیگر را نیز اضافه کرد. ایجاد این ساختار به کلاینت این امکان را می‌دهد که توابع اصلی Get و Put را به شکلی سازمان‌دهی شده و ساده برای برقراری ارتباط با سرور استفاده کند و مدیریت درخواست‌ها و پاسخ‌ها را به طور مؤثر انجام دهد.

تابع Get

تابع Get مسئول دریافت مقدار و نسخه جاری یک کلید از سرور است. این تابع از پروسه‌ی RPC برای فراخوانی متد KVServer.Get روی سرور استفاده می‌کند. کد این تابع به‌طور دائمی در تلاش است تا با سرور ارتباط برقرار کند و در صورت دریافت خطا یا عدم دریافت پاسخ، درخواست را مجدداً ارسال می‌کند. این تابع به‌صورت زیر عمل می‌کند:

□ ابتدا، ساختار GetArgs حاوی کلید مورد نظر ایجاد شده و به سرور ارسال می‌شود.

□ سپس با استفاده از تابع Call، درخواست RPC به سرور ارسال می‌شود.

□ اگر درخواست با موفقیت ارسال شد یا اگر کلید مورد نظر در سرور موجود نباشد (ErrNoKey)، حلقه خاتمه می یابد.

□ در صورت بروز خطاهای دیگر، تابع یک وقفه زمانی (time.Sleep) اعمال کرده و دوباره درخواست را ارسال می کند.

این مکانیزم retry به طور خاص برای مقابله با خطاهای موقتی و مشکلات شبکه طراحی شده است.

```

1 func (ck *Clerk) Get(key string) (string, rpc.Tversion, rpc.Err) {
2     args := rpc.GetArgs{Key: key}
3     reply := rpc.GetReply{}
4
5     for {
6         ok := ck.clnr.Call(ck.server, "KVServer.Get", &args, &reply)
7         if ok || (reply.Err == rpc.ErrNoKey) {
8             break
9         }
10        time.Sleep(100 * time.Millisecond)
11    }
12
13    return reply.Value, reply.Version, reply.Err
14 }
```

تابع Put

تابع Put مسئول به روزرسانی مقدار کلید در سرور است. برای انجام این کار، نسخه دریافتی از درخواست با نسخه فعلی کلید در سرور مقایسه می شود. اگر نسخه ها مطابقت داشته باشند، مقدار جدید نوشته می شود. در صورتی که نسخه ها مطابقت نداشته باشند، سرور باید خطای ErrVersion را برگرداند. منطق تابع به این صورت است:

□ ابتدا، ساختار PutArgs شامل کلید، مقدار و نسخه ارسال می شود.

□ درخواست RPC به سرور ارسال شده و نتیجه در ساختار PutReply ذخیره می شود.

□ اگر درخواست به موفقیت انجام نشود و خطای ErrVersion دریافت شود، تابع ErrMaybe را به برنامه بازمی‌گرداند تا نشان دهد که عملیات ممکن است در سرور انجام شده باشد ولی پاسخ گم شده است.

□ در صورت دریافت خطاهای دیگر، درخواست دوباره ارسال می‌شود.

این تابع از مکانیزم retry و backoff برای مقابله با مشکلات شبکه و گم شدن پاسخ‌ها استفاده می‌کند.

```

1 func (ck *Clerk) Put(key, value string, version rpc.Tversion) rpc.Err {
2
3     args := rpc.PutArgs{Key: key, Value: value, Version: version}
4     reply := rpc.PutReply{}
5
6     ok := ck.clnr.Call(ck.server, "KVServer.Put", &args, &reply)
7
8     for !ok {
9         ok = ck.clnr.Call(ck.server, "KVServer.Put", &args, &reply)
10        if reply.Err == rpc.ErrVersion {
11            return rpc.ErrMaybe
12        }
13        time.Sleep(100 * time.Millisecond)
14    }
15    return reply.Err
16 }

```

تابع Put و Get دو عملکرد اصلی سیستم کلید/مقدار هستند که به‌طور مستقیم با سرور ارتباط برقرار کرده و عملیات ذخیره‌سازی و بازیابی داده‌ها را انجام می‌دهند. تابع Get با استفاده از مکانیزم retry اطمینان می‌دهد که حتی در صورت بروز خطاهای موقت، درخواست به سرور ارسال می‌شود. تابع Put نیز با مدیریت نسخه‌ها و استفاده از مکانیزم ErrVersion و ErrMaybe اطمینان می‌دهد که داده‌ها به‌طور صحیح و یک‌بار در سرور ذخیره می‌شوند. این منطق و ساختار طراحی، به‌ویژه در شرایط ناپایدار شبکه، از مشکلاتی مانند گم شدن پیام‌ها، retry و اطمینان از یکسان بودن نسخه‌ها جلوگیری می‌کند و سیستم را در برابر خطاها مقاوم می‌سازد.

توضیحات ساختار کد سرور KVServer

در این بخش، به تحلیل ساختار کد سرور KVServer پرداخته می شود که به عنوان هسته اصلی سیستم کلید/مقدار طراحی و پیاده سازی شده است. این سرور وظیفه مدیریت داده های کلید/مقدار را بر عهده دارد و از مکانیزم های همزمانی برای جلوگیری از دسترسی همزمان و ناخواسته به داده ها استفاده می کند. در این توضیحات، ابتدا ساختار کلی کد بررسی می شود و سپس به تفصیل به هر یک از توابع سرور پرداخته خواهد شد. سرور پیاده سازی شده دو عملیات اصلی Get و Put را برای بازیابی و ذخیره سازی داده ها انجام می دهد و برای همزمانی و جلوگیری از مشکلات رقابتی از قفل ها mutex استفاده می کند.

ساختار کلی کد

در ابتدا، یک ساختار Record تعریف شده است که به عنوان واحد ذخیره سازی داده ها در سرور عمل می کند. این ساختار به گونه ای طراحی شده است که دو فیلد اصلی برای ذخیره سازی داده ها دارد:

□ Value: این فیلد مقدار واقعی داده ای است که برای کلید مشخص شده ذخیره می شود. مقدار داده ها به صورت رشته ذخیره می شود.

□ Version: این فیلد برای نگهداری نسخه داده استفاده می شود. هر بار که داده به روزرسانی می شود، نسخه ی آن افزایش می یابد تا از تداخل داده ها در فرآیندهای مختلف جلوگیری شود.

پس از تعریف ساختار Record، ساختار اصلی KVServer تعریف می شود که وظیفه مدیریت داده ها را بر عهده دارد. این ساختار از یک map به نام records استفاده می کند که جفت های کلید-مقدار را ذخیره می کند. map به طور مؤثر به سرور این امکان را می دهد که داده ها را ذخیره کرده و به راحتی به آن ها دسترسی پیدا کند. همچنین، برای جلوگیری از مشکلات همزمانی در دسترسی به داده ها و برای جلوگیری از دسترسی همزمان به داده ها توسط چندین کلاینت، یک قفل به نام mu از نوع sync.Mutex به این ساختار اضافه شده است.

این قفل به گونه ای عمل می کند که در هر زمان تنها یک کلاینت می تواند به داده ها دسترسی پیدا کند. از این طریق، عملیات همزمان به داده ها مدیریت شده و از مشکلات رقابتی جلوگیری می شود. در نهایت، برای ایجاد یک نمونه از سرور، تابع MakeKVServer استفاده می شود که یک نمونه جدید از KVServer را می سازد و فضای لازم برای ذخیره سازی جفت های کلید-مقدار را تخصیص می دهد.

```
1 type Record struct {
2     Value    string
3     Version  rpc.Tversion
4 }
```

```

5
6 type KVServer struct {
7     mu      sync.Mutex
8     records map[string]*Record
9 }
10
11 func MakeKVServer() *KVServer {
12     kv := &KVServer{}
13     kv.records = make(map[string]*Record)
14     return kv
15 }

```

در این کد، از قفل های sync.Mutex برای همزمانی استفاده می شود تا دسترسی به داده ها به طور ایمن و بدون تداخل انجام شود. همچنین از map برای ذخیره سازی داده ها استفاده می شود که به سرور اجازه می دهد به راحتی داده ها را ذخیره کرده و به آن ها دسترسی داشته باشد.

تابع Get

تابع Get وظیفه بازیابی مقدار و نسخه یک کلید از سرور را بر عهده دارد. این تابع ابتدا بررسی می کند که آیا کلید مورد نظر در map موجود است یا خیر. اگر کلید موجود نباشد، خطای ErrNoKey به پاسخ ارسال می شود. در غیر این صورت، مقدار و نسخه کلید به همراه خطای OK به کلاینت ارسال می شود. این کار باعث می شود که در صورت نبودن داده ی مورد نظر، پاسخ مناسبی به کلاینت ارسال شود.

عملکرد این تابع به گونه ای است که ابتدا با استفاده از mu.Lock قفل می شود تا از دسترسی همزمان به داده ها جلوگیری شود. سپس در صورتی که کلید موجود باشد، داده ها از map استخراج می شوند و به کلاینت بازمی گردند. در غیر این صورت، خطا ErrNoKey به کلاینت ارسال می شود. پس از اتمام عملیات، با استفاده از defer kv.mu.Unlock() قفل آزاد می شود.

```

1 func (kv *KVServer) Get(args *rpc.GetArgs, reply *rpc.GetReply) {
2     kv.mu.Lock()
3     defer kv.mu.Unlock()
4
5     record, ok := kv.records[args.Key]

```

```

6      if !ok {
7          reply.Err = rpc.ErrNoKey
8          return
9      }
10
11     reply.Err = rpc.OK
12     reply.Version = record.Version
13     reply.Value = record.Value
14 }

```

در این کد، از `mu.Lock` برای قفل کردن داده ها و جلوگیری از دسترسی همزمان به آن ها استفاده می شود و پس از اتمام عملیات، با `defer` قفل آزاد می شود.

تابع Put

تابع `Put` مسئول به روزرسانی یا درج مقدار جدید برای یک کلید است. این تابع ابتدا با استفاده از `mutex` برای جلوگیری از همزمانی، دسترسی به نقشه داده ها را کنترل می کند. سپس، بر اساس وضعیت نسخه ها، تصمیم می گیرد که چه عملیاتی انجام دهد:

□ اگر کلید موجود نباشد و نسخه ارسال شده صفر باشد، یک رکورد جدید با نسخه ۱ ساخته می شود.

□ اگر کلید موجود نباشد و نسخه ارسال شده غیر از صفر باشد، خطای `ErrNoKey` ارسال می شود.

□ اگر نسخه های کلید موجود و نسخه ارسال شده تطابق داشته باشد، مقدار به روزرسانی شده و نسخه آن افزایش می یابد.

□ اگر نسخه ها تطابق نداشته باشند، خطای `ErrVersion` ارسال می شود.

این طراحی باعث می شود که داده ها به صورت همزمان و بدون تضاد در سرور به روزرسانی شوند. همچنین از مکانیزم `mutex` برای هماهنگ کردن دسترسی به داده ها و جلوگیری از تضاد در تغییرات استفاده می شود.

```

1 func (kv *KVServer) Put(args *rpc.PutArgs, reply *rpc.PutReply) {
2     kv.mu.Lock()
3     defer kv.mu.Unlock()
4

```



```

5   record , exists := kv.records[args.Key]
6   switch {
7   case !exists && args.Version == 0:
8       kv.records[args.Key] = &Record{
9           Value:    args.Value ,
10          Version: 1,
11      }
12      reply.Err = rpc.OK
13
14  case !exists:
15      reply.Err = rpc.ErrNoKey
16
17  case record.Version == args.Version:
18      record.Value = args.Value
19      record.Version++
20      reply.Err = rpc.OK
21
22  default:
23      reply.Err = rpc.ErrVersion
24  }
25 }

```

در این کد، به طور واضح از `mu.Lock` و `defer kv.mu.Unlock()` استفاده می شود تا قفل تنها برای مدت زمان لازم نگه داشته شود و بعد از انجام عملیات قفل آزاد شود. همچنین با استفاده از `switch` و بررسی نسخه ها، به طور دقیق مدیریت می شود که چه زمانی مقدار کلید باید به روزرسانی شود.

تابع Kill

تابع Kill در این پیاده سازی نادیده گرفته شده است. این تابع برای متوقف کردن سرور طراحی شده است، اما چون هدف این پروژه تمرکز بر پیاده سازی اولیه سرور است و قابلیت های مربوط به شبیه سازی سرورهای افزونه ای و مکرر (replicated) مدنظر نبوده، این تابع در کد نهایی وجود ندارد.

```

1 func (kv *KVServer) Kill() {

```

```
2 }
```

تابع StartKVServer

تابع StartKVServer برای راه اندازی سرور و آماده سازی آن برای ارتباط با کلاینت ها از طریق RPC طراحی شده است. این تابع یک نمونه از KVServer می سازد و آن را به عنوان سرویس در دسترس قرار می دهد. این تابع همچنین به طور ساده سرور را برای برقراری ارتباط با کلاینت ها از طریق RPC آماده می کند و برای شروع به کار آن نیاز به هیچ گونه پارامتر یا ورودی اضافی ندارد.

```
1 func StartKVServer(ends []*labrpc.ClientEnd, gid tester.Tgid, srv int,
    persister *tester.Persister) []tester.IService {
2     kv := MakeKVServer()
3     return []tester.IService{kv}
4 }
```

در این کد، تنها یک سرور جدید ساخته می شود و به عنوان سرویس برای پذیرش درخواست های RPC آماده می شود. این تابع هیچ گونه پیچیدگی اضافی ندارد و به سادگی سرور را راه اندازی می کند. کد پیاده سازی شده برای سرور KVServer به طور مؤثر وظایف اصلی خود را برای مدیریت داده های کلید/مقدار با استفاده از مکانیزم های همزمانی انجام می دهد. این سرور از قفل ها برای جلوگیری از مشکلات همزمانی استفاده می کند و با استفاده از نسخه ها، از بروز تضاد در هنگام به روز رسانی داده ها جلوگیری می کند. طراحی ساده و کارآمد این سرور باعث می شود که بتواند به خوبی در شرایط مختلف به ویژه در شبکه های توزیع شده با قابلیت اطمینان محدود عمل کند.

توضیحات ساختار کد lock

در این بخش، قصد داریم به تحلیل ساختار کد Lock بپردازیم که به طور خاص مسئول پیاده سازی یک قفل توزیع شده برای مدیریت همزمانی دسترسی به منابع مشترک در سیستم کلید/مقدار است. در این سیستم، از سرویس کلید/مقدار استفاده می کنیم تا حالت قفل را در سرور ذخیره کرده و مدیریت کنیم. هدف اصلی این قفل این است که در شرایط همزمانی، فقط یک کلاینت در هر زمان بتواند به منابع مشترک دسترسی داشته باشد و سایر کلاینت ها باید تا آزاد شدن قفل منتظر بمانند.

ساختار کلی کد

در ابتدا، ساختار Lock تعریف شده است که شامل چندین فیلد مهم است:

□ ck: این فیلد یک اشاره گر به IKVClerk است که مسئول برقراری ارتباط با سیستم کلید/مقدار می باشد. با استفاده از این ساختار، می توانیم عملیات های Put و Get را انجام دهیم.

□ clientId: شناسه یکتای هر کلاینت است که برای شناسایی و تعیین مالک قفل به کار می رود. به عبارت دیگر، این شناسه کلاینت ها را از هم متمایز کرده و اطمینان حاصل می کند که هر کلاینت فقط قفل هایی را که خود دریافت کرده، آزاد کند.

□ lockKey: کلید منحصر به فردی است که وضعیت قفل در آن ذخیره می شود. این کلید به طور خاص برای نگهداری اطلاعات مربوط به وضعیت قفل در سرور استفاده می شود.

□ mu: این فیلد یک mutex است که برای مدیریت همزمانی و جلوگیری از مشکلات رقابتی در دسترسی به منابع مشترک مورد استفاده قرار می گیرد. این mutex از این جهت اهمیت دارد که اطمینان حاصل می کند که هیچ دو کلاینتی به طور همزمان نتوانند تغییرات همزمانی در وضعیت قفل اعمال کنند.

این ساختار وظیفه مدیریت قفل ها را بر عهده دارد و به هر کلاینت این امکان را می دهد که برای گرفتن یا آزاد کردن قفل از آن استفاده کند. به عبارت دیگر، قفل ها برای جلوگیری از دسترسی همزمان و تضاد بین درخواست ها در سیستم طراحی شده اند.

```
1 type Lock struct {
2     ck      kvtest.IKVClerk
3     clientId string
4     lockKey  string
5     mu      sync.Mutex
6 }
```

تابع MakeLock

تابع MakeLock مسئول ساخت یک نمونه جدید از Lock است. این تابع دو ورودی دریافت می کند: یک کلاینت (ck) و یک کلید (lockKey). با استفاده از این ورودی ها، یک قفل جدید ایجاد می شود. علاوه بر این، در این تابع شناسه یکتای کلاینت به طور تصادفی از طریق تابع kvtest.RandValue تولید می شود تا هر کلاینت شناسه ای منحصر به فرد برای خود داشته باشد.

این تابع تنها قفل جدید را ایجاد کرده و مقادیر اولیه را تنظیم می کند. پس از ساخت قفل، آن را برای استفاده در سایر قسمت های کد باز می گرداند.

```

1 func MakeLock(ck kvtest.IKVClerk, l string) *Lock {
2     lk := &Lock{
3         ck:      ck,
4         lockKey: l,
5         clientId: kvtest.RandValue(8),
6     }
7     return lk
8 }

```

تابع Acquire

تابع Acquire مسئول گرفتن قفل از سرور است. در این تابع، ابتدا بررسی می شود که آیا قفل در حال حاضر متعلق به کلاینت است یا خیر. اگر قفل در دسترس باشد (یعنی قفل قبلاً گرفته نشده باشد یا برای این کلاینت باشد)، کلاینت می تواند به راحتی قفل را بگیرد. در غیر این صورت، کلاینت باید منتظر بماند تا قفل توسط کلاینت دیگر آزاد شود.

در این تابع، ابتدا با استفاده از تابع Get وضعیت قفل بررسی می شود. اگر مقدار موجود در قفل برابر با شناسه کلاینت باشد، یعنی قفل قبلاً توسط این کلاینت گرفته شده است و نیازی به تغییر آن نیست. در غیر این صورت، تابع سعی می کند با استفاده از تابع Put شناسه خود را به عنوان صاحب قفل در سرور ذخیره کند.

همچنین برای جلوگیری از تداخل دسترسی به منابع و مشکلات همزمانی، از mutex استفاده شده است. این mutex به طور مؤثر دسترسی به منابع را مدیریت می کند و اطمینان حاصل می کند که فقط یک کلاینت در هر زمان تغییرات روی قفل را انجام دهد.

در صورت بروز هرگونه خطا (مانند گم شدن پیام ها یا تأخیر در پردازش)، تابع به طور خودکار تلاش مجدد انجام می دهد و تا زمانی که قفل به دست نیامده باشد، عملیات را تکرار می کند.

```

1 func (lk *Lock) Acquire() {
2     i := 100
3     for {
4         lk.mu.Lock()
5         val, version, err := lk.ck.Get(lk.lockKey)
6

```

```

7      if val == lk.clientId {
8          lk.mu.Unlock()
9          return
10     }
11
12     if err == rpc.ErrNoKey || (err == rpc.OK && val == "") {
13         err = lk.ck.Put(lk.lockKey, lk.clientId, version)
14         if err == rpc.OK {
15             lk.mu.Unlock()
16             return
17         }
18     }
19
20     lk.mu.Unlock()
21     time.Sleep(time.Duration(i) * time.Millisecond)
22     i += 10
23 }
24 }

```

در اینجا، علاوه بر استفاده از mutex برای مدیریت همزمانی، از retry و backoff نیز استفاده می‌شود. این مکانیزم به ما کمک می‌کند که در صورت مواجهه با مشکلات شبکه، بتوانیم درخواست را دوباره ارسال کرده و از بروز خطاهای موقتی جلوگیری کنیم.

تابع Release

تابع Release برای آزاد کردن قفل از سرور استفاده می‌شود. در اینجا، ابتدا با استفاده از تابع Get بررسی می‌شود که آیا قفل به این کلاینت تعلق دارد یا خیر. اگر قفل متعلق به کلاینت باشد، با استفاده از Put مقدار کلید به صورت خالی ذخیره می‌شود تا قفل آزاد شود. در غیر این صورت، تابع تلاش می‌کند که دوباره عملیات Release را انجام دهد. مانند تابع Acquire، در اینجا نیز از mutex برای همزمانی استفاده شده است تا اطمینان حاصل شود که هیچ‌گونه تضاد یا تداخل در دسترسی به منابع وجود ندارد. در این تابع هم از مکانیزم retry برای مقابله با مشکلات احتمالی مانند گم شدن پیام‌ها یا تأخیرهای شبکه استفاده می‌شود.

```
1 func (lk *Lock) Release() {  
2     i := 100  
3     for {  
4         lk.mu.Lock()  
5         value, version, err := lk.ck.Get(lk.lockKey)  
6  
7         if err == rpc.OK && value == lk.clientId {  
8             lk.ck.Put(lk.lockKey, "", version)  
9             lk.mu.Unlock()  
10            break  
11        }  
12  
13        lk.mu.Unlock()  
14        time.Sleep(time.Duration(i) * time.Millisecond)  
15        i += 10  
16    }  
17 }
```

کد قفل توزیع شده ای که طراحی کرده ایم به طور مؤثر و کارآمد مدیریت قفل ها را در یک سیستم توزیع شده با استفاده از سیستم کلید/مقدار انجام می دهد. این سیستم به گونه ای طراحی شده که در برابر خطاهای شبکه و همزمانی دسترسی به منابع از طریق مکانیزم های retry و backoff مقاوم باشد. به طور کلی، این سیستم به هر کلاینت این امکان را می دهد که به صورت ایمن و هماهنگ به منابع مشترک دسترسی داشته باشد و در شرایطی که چندین کلاینت بخواهند به یک منبع دسترسی داشته باشند، از بروز تضاد و تداخل جلوگیری می کند.

این طراحی ساده اما مؤثر از روش های قفل توزیع شده و همزمانی برای جلوگیری از مشکلات همزمانی و تداخل در دسترسی به منابع استفاده می کند. به ویژه در شرایط ناپایدار شبکه، این سیستم با استفاده از مکانیزم های بازفرستادن درخواست (retry) و تأخیر (backoff) توانسته است عملکرد خود را به طور صحیح و بدون هیچ گونه مشکل حفظ کند. این سیستم علاوه بر ساده بودن، بسیار مؤثر است و توانایی دارد در برابر خطاهای احتمالی و شرایط رقابتی پیچیده ای که در محیط های توزیع شده وجود دارد، ایمن باقی بماند.

توضیحات ساختار کد TestKV

در این بخش به تحلیل ساختار کد مربوط به اجرای تست های سامانه کلید/مقدار (Key/Value) می پردازیم. این بخش از کد وظیفه راه اندازی، مدیریت و اجرای تست ها در دو حالت شبکه قابل اطمینان و ناپایدار را بر عهده دارد و به منظور ارزیابی عملکرد سیستم کلید/مقدار طراحی شده است.

ساختار کلی کد

در ابتدا ساختار TestKV تعریف شده که شامل سه فیلد اصلی است:

□ kvtest.Test: اشاره گری به ساختار Test که نمایانگر مجموعه ای از ابزارها و داده های مرتبط با اجرای یک تست کلی است.

□ testing.T t: اشاره گری به ساختار testing.T در زبان Go که برای مدیریت و گزارش نتایج تست ها به کار می رود.

□ bool reliable: متغیری بولی که تعیین می کند تست ها در محیط شبکه قابل اطمینان true یا ناپایدار false اجرا شوند.

این ساختار به عنوان لایه ای مدیریتی برای تنظیم و کنترل تست ها عمل کرده و اطلاعات مربوط به نوع تست و محیط اجرای آن را ذخیره می کند.

```
1 type TestKV struct {  
2     *kvtest.Test  
3     t          *testing.T  
4     reliable bool  
5 }
```

تابع MakeTestKV

این تابع مسئول ایجاد یک نمونه جدید از TestKV است که برای اجرای تست ها به کار می رود. روند عملکرد این تابع به شرح زیر است:

□ با فراخوانی تابع tester.MakeConfig، یک کانفیگ برای تست ساخته می شود که تعداد سرورها، نوع شبکه (قابل اطمینان یا ناپایدار) و تابع شروع سرور را دریافت می کند.

□ سپس یک نمونه جدید از TestKV ساخته شده و فیلدهای t و reliable مقداردهی می‌شوند.

□ در نهایت با فراخوانی kvtest.MakeTest تست اصلی ایجاد شده و به فیلد Test اختصاص داده می‌شود تا این ساختار بتواند به تمام امکانات اجرای تست‌ها دسترسی داشته باشد.

□ نمونه ایجاد شده بازگردانده می‌شود تا در اجرای تست‌ها استفاده شود.

```

1 func MakeTestKV(t *testing.T, reliable bool) *TestKV {
2     cfg := tester.MakeConfig(t, 1, reliable, StartKVServer)
3     ts := &TestKV{
4         t:      t,
5         reliable: reliable,
6     }
7     ts.Test = kvtest.MakeTest(t, cfg, false, ts)
8     return ts
9 }

```

تابع MakeClerk

این تابع برای ساخت یک کلاینت جدید در چارچوب تست به کار می‌رود. کلاینت وظیفه ارسال درخواست‌های Put و Get را به سرور کلید/مقدار بر عهده دارد. نحوه عملکرد آن به شرح زیر است:

□ ابتدا با استفاده از کانفیگ موجود در TestKV، کلاینتی جدید ساخته می‌شود که قادر به برقراری ارتباط با سرور است.

□ سپس با فراخوانی تابع MakeClerk، یک کلاینت سطح بالاتر ساخته می‌شود که آدرس مشخصی از سرور (سرور شماره صفر در گروه صفر) به آن داده شده است.

□ این کلاینت ساخته شده در قالب یک TestClerk بسته‌بندی شده و به عنوان خروجی بازگردانده می‌شود تا در تست‌ها مورد استفاده قرار گیرد.

```

1 func (ts *TestKV) MakeClerk() kvtest.IKVClerk {
2     clnt := ts.Config.MakeClient()
3     ck := MakeClerk(clnt, tester.ServerName(tester.GRP0, 0))

```



```

4     return &kvtest.TestClerk{ck, clnt}
5 }

```

تابع DeleteClerk

در پایان استفاده از کلاینت، این تابع برای حذف کلاینت ایجاد شده و آزادسازی منابع مربوطه استفاده می شود. عملکرد آن شامل:

□ تبدیل نمونه کلاینت عمومی (IKVClerk) به نوع اختصاصی TestClerk انجام می شود تا بتوان به ساختار داخلی آن دسترسی داشت.

□ سپس کلاینت با فراخوانی تابع DeleteClient از سیستم حذف و منابع مرتبط آزاد می شوند.

این کار باعث می شود که کلاینت ها به صورت منظم مدیریت شده و از نشت منابع جلوگیری شود.

```

1 func (ts *TestKV) DeleteClerk(ck kvtest.IKVClerk) {
2     tck := ck.(*kvtest.TestClerk)
3     ts.DeleteClient(tck.Cln)
4 }

```

نتایج اجرای تست های خودکار

این تست ها شامل بررسی عملکرد سیستم در شرایط شبکه قابل اطمینان و ناپایدار در دو بخش اصلی هستند: تست های سرویس کلید/مقدار و تست های قفل توزیع شده.

تست های سرویس کلید/مقدار

۱. تست سرویس کلید/مقدار در شرایط شبکه قابل اطمینان (TestOneClientReliable و TestManyClientsReliable)

این تست ها اطمینان حاصل می کنند که سرویس کلید/مقدار در شرایط شبکه قابل اطمینان به درستی عمل می کند. در این شرایط، شبکه به طور پایدار عمل می کند و پیام ها بدون مشکل به سرور ارسال می شوند و پاسخ ها به درستی برگشت داده می شوند. هدف از این تست ها بررسی موارد زیر است:

□ درخواست های Put و Get: این تست ها بررسی می کنند که آیا سرور به درستی درخواست های Put و Get را پردازش کرده و داده ها را ذخیره یا بازیابی می کند.

□ ذخیره سازی و بازیابی داده ها: اطمینان از اینکه داده ها به درستی ذخیره و بازیابی می شوند، یعنی پس از ذخیره سازی یک مقدار توسط کلاینت، هنگام درخواست از سرور همان مقدار برگشت داده شود.

□ ارتباطات صحیح میان کلاینت و سرور: در این تست ها، رفتار ارتباطی میان کلاینت ها و سرور در شرایط پایدار شبکه بررسی می شود و از صحت برقراری ارتباط اطمینان حاصل می شود.

۲. تست سرویس کلید/مقدار در شرایط شبکه ناپایدار (TestOneClientUnreliable و TestManyClientsUnreliable)

در این تست ها، هدف ارزیابی عملکرد سیستم در شرایط شبکه ناپایدار است که ممکن است شامل گم شدن پیام ها، تأخیر یا قطع ارتباط های موقتی باشد. در این تست ها موارد زیر مورد بررسی قرار می گیرند:

□ مکانیزم retry و backoff: بررسی می شود که آیا سیستم می تواند در صورت بروز مشکلات موقتی شبکه، درخواست ها را مجدداً ارسال کرده و از آسیب های احتمالی جلوگیری کند.

□ مدیریت خطاها: ارزیابی عملکرد سیستم در مواجهه با خطاهای شبکه مانند گم شدن پیام ها و تأخیر در پاسخ ها، و همچنین بررسی بازگشت خطاهای مناسب از سرور.

□ اطمینان از ذخیره سازی و بازیابی داده ها: اطمینان از اینکه حتی در صورت بروز مشکلات شبکه، سیستم قادر به ذخیره سازی داده ها و بازیابی درست آنها است.

تست های قفل توزیع شده

۳. تست قفل در شرایط شبکه قابل اطمینان (TestOneClientReliable و TestManyClientsReliable)

در این تست ها، هدف بررسی عملکرد قفل توزیع شده در شرایط شبکه قابل اطمینان است. در این تست ها، چندین کلاینت همزمان تلاش می کنند که قفل را به دست آورده و آن را آزاد کنند. این تست ها اطمینان می دهند که:

□ دسترس پذیری قفل: فقط یک کلاینت می تواند در هر زمان خاص قفل را به دست آورد و سایر کلاینت ها باید منتظر آزاد شدن قفل شوند.

□ عملیات صحیح پس از به دست آوردن قفل: پس از به دست آوردن قفل توسط یک کلاینت، این کلاینت باید بتواند عملیات Put و Get را انجام دهد و داده ها به درستی ذخیره و بازیابی شوند.

□ آزاد کردن قفل: پس از اتمام عملیات، کلاینت باید بتواند قفل را آزاد کند و قفل به سایر کلاینت ها دسترسی بدهد.

۴. تست قفل در شرایط شبکه ناپایدار (TestOneClientUnreliable و TestManyClientsUnreliable)

این تست ها مشابه تست های قبلی هستند، اما در شرایط شبکه ناپایدار اجرا می شوند. در این تست ها، موارد زیر بررسی می شوند:

□ مدیریت همزمانی در شرایط ناپایدار: بررسی اینکه آیا سیستم قفل قادر است در شرایط ناپایدار شبکه، همزمانی دسترسی به منابع را به درستی مدیریت کند و از ایجاد شرایط رقابتی جلوگیری کند.

□ آزاد کردن قفل پس از قطع ارتباط: اطمینان از اینکه حتی در صورت قطع ارتباط یا گم شدن پیام ها، قفل به درستی آزاد شده و سایر کلاینت ها می توانند به آن دسترسی داشته باشند.

□ بررسی مکانیزم های retry و backoff: ارزیابی اینکه آیا در صورت بروز مشکلات موقتی در شبکه، کلاینت ها می توانند دوباره تلاش کنند و قفل را به درستی آزاد کنند.

نتیجه گیری کلی از تست ها

این چهار نوع تست به طور جامع عملکرد سیستم را در شرایط مختلف شبکه ارزیابی می کنند. از یک سو، تست های سرویس کلید/مقدار بررسی می کنند که آیا سیستم به درستی داده ها را ذخیره و بازیابی می کند و قادر به مدیریت خطاهای شبکه است. از سوی دیگر، تست های قفل توزیع شده به طور خاص بر رفتار سیستم در مواجهه با دسترسی همزمان به منابع و نحوه مدیریت آن ها تمرکز دارند.

در نهایت، این تست ها به ویژه در مواجهه با شرایط ناپایدار شبکه، عملکرد صحیح سیستم را تضمین می کنند و نشان می دهند که چگونه سیستم می تواند از مشکلات شبکه جلوگیری کرده و داده ها را به طور صحیح مدیریت کند.

اجرای تست ها و نتایج

برای ارزیابی عملکرد سیستم، از مجموعه ای از تست های خودکار استفاده کردیم که به بررسی رفتار سیستم در شرایط مختلف شبکه می پرداختند. این تست ها به ویژه در شرایط شبکه قابل اطمینان و ناپایدار اجرا شدند تا اطمینان حاصل شود که سیستم به درستی در هر دو حالت کار می کند.

نحوه اجرای تست‌ها

برای اجرای تست‌ها، ابتدا از فایل‌های تست موجود در پروژه استفاده کرده و آن‌ها را با استفاده از دستور `test go` اجرا کردیم. تست‌ها در دو حالت اصلی اجرا شدند:

□ شبکه قابل اطمینان: در این حالت، شبکه به‌طور پایدار عمل می‌کند و پیام‌ها بدون تاخیر یا گم شدن به سرور ارسال می‌شوند.

□ شبکه ناپایدار: در این حالت، مشکلاتی مانند گم شدن پیام‌ها، تأخیر در ارسال یا قطع ارتباط‌ها وجود دارد.

برای هر یک از این حالت‌ها، چهار نوع تست اجرا شد که شامل تست‌های مربوط به سرویس کلید/مقدار و قفل توزیع شده بودند. این تست‌ها به‌طور خودکار از طریق محیط آزمایش (testing) در Go اجرا شدند.

بررسی شرایط رقابتی (Race Condition) و اهمیت اجرای تست‌های race

یکی از مهم‌ترین چالش‌ها در توسعه سیستم‌های همزمان و توزیع شده، مواجهه با شرایط رقابتی یا `race condition` است. شرایط رقابتی زمانی رخ می‌دهد که چندین فرآیند به صورت همزمان و بدون هماهنگی مناسب به داده‌ها یا منابع مشترک دسترسی پیدا کنند و این باعث شود که ترتیب دسترسی و تغییرات بر روی داده‌ها غیرقابل پیش‌بینی و ناسازگار شود. چنین وضعیتی می‌تواند منجر به بروز خطاهای پیچیده، عدم پایداری سیستم و رفتارهای نادرست شود. برای اطمینان از اینکه سیستم ما در برابر چنین شرایطی مقاوم است، علاوه بر اجرای تست‌های معمول، تمامی تست‌ها را با فعال کردن فلگ `race` در زبان Go نیز اجرا کردیم. این فلگ ابزارهایی را فعال می‌کند که به صورت خودکار شرایط رقابتی احتمالی در زمان اجرای برنامه را شناسایی و گزارش می‌دهند.

اجرای تست‌ها در دو حالت (بدون فلگ `race` و با فلگ `race`) به ما کمک کرد تا:

□ مشکلات و خطاهای ناشی از دسترسی همزمان نامناسب به منابع مشترک را شناسایی کنیم.

□ اطمینان حاصل کنیم که کد ما به درستی از قفل‌ها و مکانیزم‌های همزمانی استفاده می‌کند و شرایط رقابتی ایجاد نمی‌شود.

□ عملکرد صحیح سیستم در شرایط واقعی و پیچیده‌ای که همزمانی بالایی وجود دارد را تایید کنیم.

خوشبختانه در هر دو حالت اجرا، تمامی چهار سری تست (شامل تست‌های سرویس کلید/مقدار و قفل توزیع شده در شبکه‌های قابل اطمینان و ناپایدار) به طور کامل پاس شدند و هیچ نشانه‌ای از خطاهای مربوط به شرایط رقابتی مشاهده نشد. این موضوع نشان‌دهنده طراحی دقیق و استفاده صحیح از قفل‌ها و مکانیزم‌های همزمانی در پروژه است و اطمینان بالایی از پایداری و صحت عملکرد سیستم فراهم می‌کند.

نتایج تست ها

برای هر نوع تست، دو تصویر نتایج ارائه شده است: یک تصویر مربوط به اجرای معمول تست ها بدون فلگ race و یک تصویر مربوط به اجرای تست ها با فعال بودن فلگ race نمایش داده شده است. این روش به منظور بررسی جامع تر و اطمینان از عدم وجود مشکلات شرایط رقابتی انجام شده است.

```
PS E:\elahe\git\Distributed-Systems-S04\Project 2\6.5840\src\kvsrv1> go test -v -run Reliable
=== RUN TestReliablePut
One client and reliable Put (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs 5 #Ops 0
--- PASS: TestReliablePut (0.00s)
=== RUN TestPutConcurrentReliable
Test: many clients racing to put values to the same key (reliable network)...
info: linearizability check timed out, assuming history is ok
... Passed -- time 11.0s #peers 1 #RPCs 56747 #Ops 56747
--- PASS: TestPutConcurrentReliable (11.04s)
=== RUN TestMemPutManyClientsReliable
Test: memory use many put clients (reliable network)...
... Passed -- time 9.4s #peers 1 #RPCs 100000 #Ops 0
--- PASS: TestMemPutManyClientsReliable (17.16s)
PASS
ok      6.5840/kvsrv1    29.317s
```

شکل ۱: خروجی نتایج تست های سرویس کلید/مقدار در شرایط شبکه قابل اطمینان (بدون فلگ race)

```
fereshte@asus:~/Distributed-Systems-S04/Project 2/6.5840/src/kvsrv1$ go test -race -v -run Reliable
=== RUN TestReliablePut
One client and reliable Put (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs 5 #Ops 0
--- PASS: TestReliablePut (0.00s)
=== RUN TestPutConcurrentReliable
Test: many clients racing to put values to the same key (reliable network)...
... Passed -- time 7.9s #peers 1 #RPCs 12915 #Ops 12915
--- PASS: TestPutConcurrentReliable (7.91s)
=== RUN TestMemPutManyClientsReliable
Test: memory use many put clients (reliable network)...
... Passed -- time 32.3s #peers 1 #RPCs 100000 #Ops 0
--- PASS: TestMemPutManyClientsReliable (63.91s)
PASS
ok      6.5840/kvsrv1    72.849s
```

شکل ۲: خروجی نتایج تست های سرویس کلید/مقدار در شرایط شبکه قابل اطمینان (با فلگ race)

```
PS E:\elahe\git\Distributed-Systems-S04\Project 2\6.5840\src\kvsrv1\lock> go test -v -run Reliable
FAIL    6.5840/kvsrv1/lock 27.009s
=== RUN TestOneClientReliable
Test: 1 lock clients (reliable network)...
... Passed -- time 2.0s #peers 1 #RPCs 1282 #Ops 0
--- PASS: TestOneClientReliable (2.01s)
=== RUN TestManyClientsReliable
Test: 10 lock clients (reliable network)...
... Passed -- time 2.1s #peers 1 #RPCs 104092 #Ops 0
--- PASS: TestManyClientsReliable (2.10s)
PASS
ok      6.5840/kvsrv1/lock 4.600s
```

شکل ۳: خروجی نتایج تست های سرویس کلید/مقدار در شرایط شبکه ناپایدار (بدون فلگ race)

```

● fereshte@asus:~/Distributed-Systems-S04/Project 2/6.5840/src/kvsrv1$ go test -race -v
=== RUN   TestReliablePut
One client and reliable Put (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs 5 #Ops 0
--- PASS: TestReliablePut (0.00s)
=== RUN   TestPutConcurrentReliable
Test: many clients racing to put values to the same key (reliable network)...
... Passed -- time 6.0s #peers 1 #RPCs 11003 #Ops 11003
--- PASS: TestPutConcurrentReliable (6.01s)
=== RUN   TestMemPutManyClientsReliable
Test: memory use many put clients (reliable network)...
... Passed -- time 34.6s #peers 1 #RPCs 100000 #Ops 0
--- PASS: TestMemPutManyClientsReliable (67.53s)
=== RUN   TestUnreliableNet
One client (unreliable network)...
... Passed -- time 7.5s #peers 1 #RPCs 258 #Ops 209
--- PASS: TestUnreliableNet (7.55s)
PASS
ok      6.5840/kvsrv1  82.123s

```

شکل ۴: خروجی نتایج تست های سرویس کلید/مقدار در شرایط شبکه ناپایدار (با فلگ race)

```

PS E:\elahe\git\Distributed-Systems-S04\Project 2\6.5840\src\kvsrv1> go test -v
● === RUN   TestReliablePut
One client and reliable Put (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs 5 #Ops 0
--- PASS: TestReliablePut (0.00s)
=== RUN   TestPutConcurrentReliable
Test: many clients racing to put values to the same key (reliable network)...
info: linearizability check timed out, assuming history is ok
... Passed -- time 11.0s #peers 1 #RPCs 60635 #Ops 60635
--- PASS: TestPutConcurrentReliable (11.03s)
=== RUN   TestMemPutManyClientsReliable
Test: memory use many put clients (reliable network)...
... Passed -- time 7.6s #peers 1 #RPCs 100000 #Ops 0
--- PASS: TestMemPutManyClientsReliable (12.28s)
=== RUN   TestUnreliableNet
One client (unreliable network)...
... Passed -- time 5.8s #peers 1 #RPCs 250 #Ops 212
--- PASS: TestUnreliableNet (5.84s)
PASS
ok      6.5840/kvsrv1  30.077s

```

شکل ۵: خروجی نتایج تست های قفل توزیع شده در شرایط شبکه قابل اطمینان (بدون فلگ race)


```

• fereshte@asus:~/Distributed-Systems-S04/Project 2/6.5840/src/kvsrv1/lock$ go test -race -v -run Reliable
=== RUN   TestOneClientReliable
Test: 1 lock clients (reliable network)...
... Passed -- time 2.0s #peers 1 #RPCs 1079 #Ops 0
--- PASS: TestOneClientReliable (2.01s)
=== RUN   TestManyClientsReliable
Test: 10 lock clients (reliable network)...
... Passed -- time 4.2s #peers 1 #RPCs 1355 #Ops 0
--- PASS: TestManyClientsReliable (4.24s)
PASS
ok      6.5840/kvsrv1/lock      7.256s

```

شکل ۶: خروجی نتایج تست های قفل توزیع شده در شرایط شبکه قابل اطمینان (با فلگ race)

```

PS E:\elaha\git\Distributed-Systems-S04\Project 2\6.5840\src\kvsrv1\lock> go test -v
=== RUN   TestOneClientReliable
Test: 1 lock clients (reliable network)...
... Passed -- time 2.0s #peers 1 #RPCs 1289 #Ops 0
--- PASS: TestOneClientReliable (2.01s)
=== RUN   TestManyClientsReliable
Test: 10 lock clients (reliable network)...
... Passed -- time 3.0s #peers 1 #RPCs 1565 #Ops 0
--- PASS: TestManyClientsReliable (2.98s)
=== RUN   TestOneClientUnreliable
Test: 1 lock clients (unreliable network)...
... Passed -- time 2.1s #peers 1 #RPCs 79 #Ops 0
--- PASS: TestOneClientUnreliable (2.12s)
=== RUN   TestManyClientsUnreliable
Test: 10 lock clients (unreliable network)...
... Passed -- time 4.0s #peers 1 #RPCs 403 #Ops 0
--- PASS: TestManyClientsUnreliable (4.03s)
PASS
ok      6.5840/kvsrv1/lock      11.605s

```

شکل ۷: خروجی نتایج تست های قفل توزیع شده در شرایط شبکه ناپایدار (بدون فلگ race)

```

• fereshte@asus:~/Distributed-Systems-S04/Project 2/6.5840/src/kvsrv1/lock$ go test -race -v
=== RUN   TestOneClientReliable
Test: 1 lock clients (reliable network)...
... Passed -- time 2.0s #peers 1 #RPCs 1121 #Ops 0
--- PASS: TestOneClientReliable (2.01s)
=== RUN   TestManyClientsReliable
Test: 10 lock clients (reliable network)...
... Passed -- time 4.2s #peers 1 #RPCs 1374 #Ops 0
--- PASS: TestManyClientsReliable (4.24s)
=== RUN   TestOneClientUnreliable
Test: 1 lock clients (unreliable network)...
... Passed -- time 2.3s #peers 1 #RPCs 92 #Ops 0
--- PASS: TestOneClientUnreliable (2.34s)
=== RUN   TestManyClientsUnreliable
Test: 10 lock clients (unreliable network)...
... Passed -- time 4.7s #peers 1 #RPCs 351 #Ops 0
--- PASS: TestManyClientsUnreliable (4.72s)
PASS
ok      6.5840/kvsrv1/lock      14.317s

```

شکل ۸: خروجی نتایج تست های قفل توزیع شده در شرایط شبکه ناپایدار (با فلگ race)

نتیجه گیری

پس از اجرای تمام تست‌ها، نتایج نشان داد که سیستم به درستی در شرایط مختلف شبکه عمل می‌کند:

□ در شرایط شبکه قابل اطمینان، سیستم به‌طور صحیح داده‌ها را ذخیره و بازیابی کرده و قفل‌ها به درستی بین کلاینت‌ها توزیع شدند.

□ در شرایط ناپایدار شبکه، سیستم توانست با استفاده از مکانیزم `retry` و `backoff` به درستی از مشکلات ناشی از گم شدن پیام‌ها و تأخیرها جلوگیری کند و عملکرد خود را حفظ کند.

□ تست‌های قفل توزیع‌شده نشان داد که سیستم قادر است به‌طور ایمن دسترسی به منابع مشترک را مدیریت کرده و از تداخل در دسترسی‌های همزمان جلوگیری کند.

این تست‌ها به‌ویژه در ارزیابی رفتار سیستم در شرایط مختلف شبکه و شبیه‌سازی شرایط واقعی شبکه مفید بودند. با استفاده از این تست‌ها، به‌طور دقیق از صحت عملکرد سیستم در شرایط مختلف اطمینان حاصل کردیم و توانستیم نقاط قوت و ضعف سیستم را شناسایی کنیم. در نهایت، سیستم طراحی‌شده قادر است در برابر چالش‌های محیط‌های ناپایدار شبکه به‌طور موثر عمل کند.